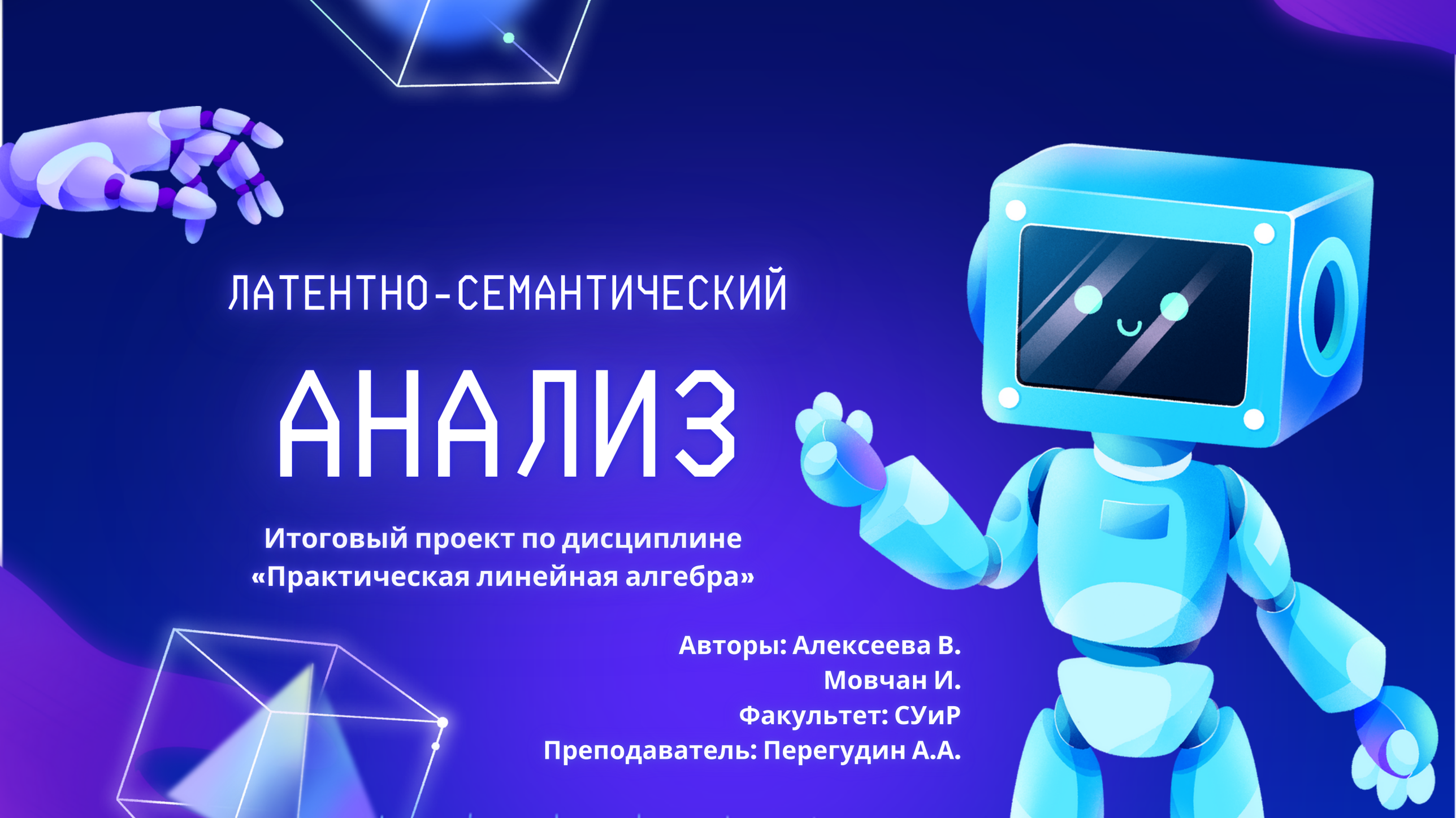


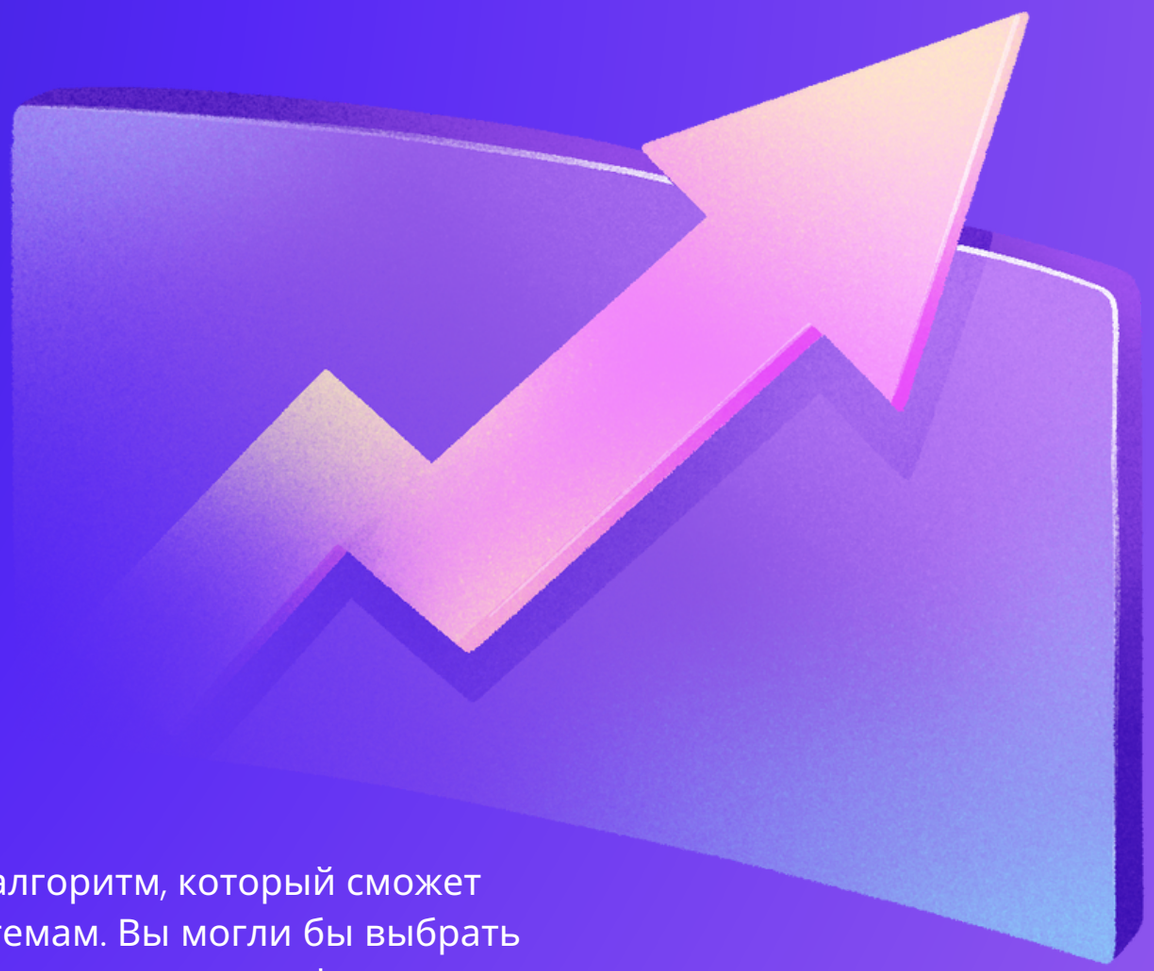
ЛАТЕНТНО-СЕМАНТИЧЕСКИЙ АНАЛИЗ

Итоговый проект по дисциплине
«Практическая линейная алгебра»

Авторы: Алексеева В.
Мовчан И.
Факультет: СУиР
Преподаватель: Перегудин А.А.

ВВЕДЕНИЕ

На форумах по программированию часто спрашивают, как найти тексты с похожим смыслом и какие алгоритмы для этого существуют? Один из подходов, используемых поисковыми гигантами и SEO-специалистами, называется Latent Semantic Analysis (LSA). Этот вариант очень эффективен и предполагает использование алгоритмов для выявления **закономерностей и взаимосвязей** в больших массивах данных, что позволяет получить более точные результаты поиска.



В чем заключается трудность анализа?

Предположим, перед вами поставлена задача создать алгоритм, который сможет отличать новостные статьи по представленным в них темам. Вы могли бы выбрать слова, уникальные для каждой категории, и использовать их для классификации статей. Однако такой подход имеет трудности: сложно определить, как обрабатывать статьи, содержащие **слова из нескольких категорий**. Кроме того, дело усложняют **омонимы**, т.е. слова с несколькими значениями. Например, слово «банки» может относиться к стеклянным сосудам или к финансовым учреждениям.

МЕТОДИКА И ЕЕ ДОПУЩЕНИЯ:

Документы рассматриваются как набор слов. Порядок слов в документах не имеет значения, он полностью игнорируется. Единственное, самым важным аспектом является **частота появления слова** в документе.

Семантическое значение документа определяется набором слов, которые имеют тенденцию встречаться вместе. Например, в отчетах о состоянии фондового рынка слова «фонд», «акции» и «доллар» **часто пересекаются**.

Предполагается, что каждое слово имеет только **одно значение**. Хотя это и сильное упрощение, но оно делает проблему анализа решаемой.

Основным математическим инструментом, используемым в LSA, является **SVD**, с которым мы уже знакомы.

ПРИМЕР РАБОТЫ LSA В ДЕЙСТВИИ!

Выберем новостные заголовки из файла data.txt.

Далее избавимся от **постороннего шума**: уберем пунктуацию, союзы, предлоги и частицы.
Далее необходимо привести слова к их **начальной форме**, чтобы не возникали противоречия с одинаковыми по смыслу, но разными по форме выражениями, омонимами.

Для решения этой задачи мы решили использовать библиотеку **pymystem3** от Яндекса, но можно воспользоваться pymorphy2; опыты показывают, что ведут они себя примерно одинаково).

Для многократного улучшения вычислительных способностей убираем **уникальные для каждого предложения слова**.

SpaceX вывела на орбиту партию **спутник**ов Starlink
SpaceX сообщила о запуске ракеты с секретным шаттлом США
Спутник Метеор-М доставили на Восточный
Астрономы провели наблюдения космического объекта неизвестного происхождения
Tesla перевыпустит автомобиль Model Y
В Xiaomi заявили о конкуренции с **Tesla**
Обнаружены четыре двойных **бел**ых **карлик**а
Загадка **бел**ых **карлик**ов. Российские ученые объяснили малую массу звезд
Российские **ученые** поняли, как грибной яд работает против бактерий
Ученые выяснили, как **рак**овая опухоль по-соседски заражает здоровые клетки
Биолог назвала продукты, которые могут помочь в борьбе с **рак**ом
В **Росс**ии повысили точность **диагностик**и **рак**а **кож**и с помощью нейросети
Ученые подобрали лазер для **диагностик**и **рак**а через кожу

РАБОТА С КОДОМ

ИМПОРТ 01

```
import nltk
import string
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import re
from sklearn.cluster import KMeans
from pymystem3 import Mystem
import os.path
```

ФУНКЦИЯ 02

```
def main():
    filename = input("Dataset file: ")
    while (not os.path.isfile(filename)):
        print("File hasn't found, try again")
        filename = input("Dataset file: ")
    k = int(input("Dimension(preferably 1<=k<=3 to show graphics): "))
    while (not k>=1):
        print("k should be >= 1, try again")
        k = int(input("Dimension(preferably 1<=k<=3 to show graphics): "))
    n = int(input("Number of clusters: "))
    while (not n >= 1):
        print("n should be >= 1, try again")
        n = int(input("Number of clusters: "))
    # initial data from file
    with open(filename) as file:
        sents = (line.rstrip("\n") for line in file.readlines())
```

ФИЛЬТР ПРЕДЛОЖЕНИЙ

```
new_sents = []
stop_words = nltk.corpus.stopwords.words("russian")
m = Mystem()
table = str.maketrans("", "", string.punctuation)
counts = {}
for sent in sents:
    sent = sent.translate(table) # removing punctuation
    new_sent = "".join(m.lemmatize(sent)) # normal form of words
    tokenized_sent = nltk.tokenize.word_tokenize(new_sent, language="russian") # tokenize
    filtered_sent = []
    for token in tokenized_sent:
        token = token.lower()
        if token not in stop_words:
            filtered_sent.append(token)
            counts[token] = counts.get(token, 0) + 1 # counting
    new_sents.append(filtered_sent)
counts = dict(filter(lambda pair: pair[1] > 1, counts.items())) # word count > 1
final_sents=[]
for sent in new_sents:
    new_sent = [word for word in sent if word in counts] # removing unique words
    final_sents.append(new_sent)

# output: final sentences
print("Processed sentences:")
for i in range(len(final_sents)):
    print(f"S{i+1}:", " ".join(final_sents[i]))
```

ПРЕОБРАЗОВАННЫЕ ПРЕДЛОЖЕНИЯ

```
S1: spacex спутник
S2: spacex
S3: спутник
S4:
S5: tesla
S6: tesla
S7: белый карлик
S8: белый карлик российский ученый
S9: российский ученый
S10: ученый
S11: рак
S12: диагностика рак кожа
S13: ученый диагностика рак кожа
```

СОСТАВЛЕНИЕ МАТРИЦЫ И СИНГУЛЯРНОЕ РАЗЛОЖЕНИЕ

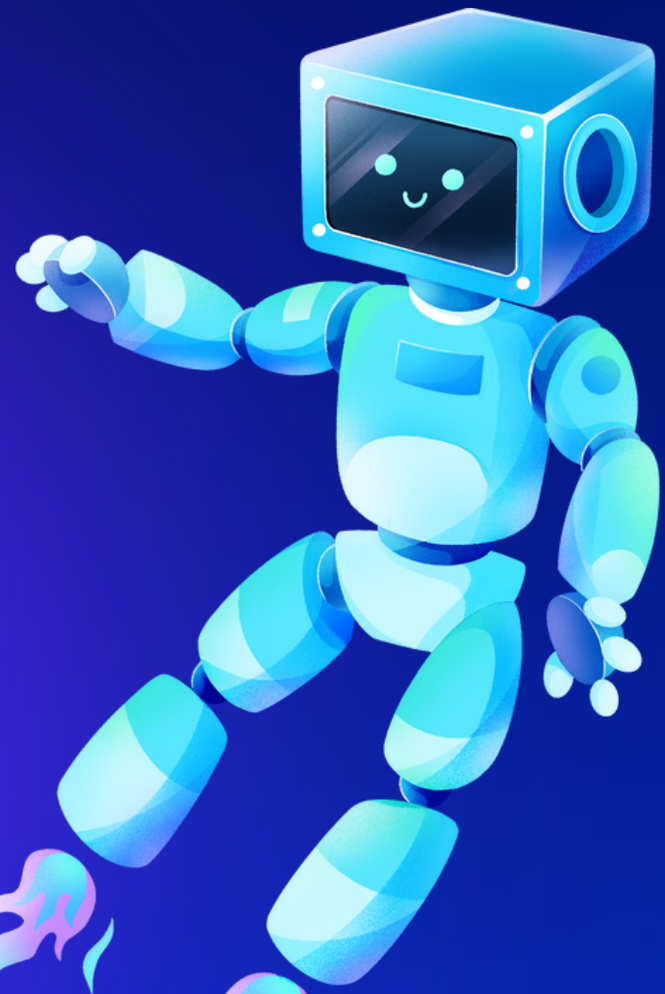
Перейдем к составлению **матрицы**, в которой в качестве строк будут выступать слова, найденные из предыдущего шага, а в качестве столбцов - предложения.

Элемент a_ij матрицы A есть количество вхождений i-го слова в j-ое предложение.
Для составленной матрицы проводим **сингулярное разложение** с помощью библиотеки numpy, получая матрицы U, E и VT.
Отметим, что **сингулярные числа отсортированы по убыванию**.

```
matrix = []
    words = list(sorted(counts.keys()))
    for word in words:
        row = []
        for sent in final_sents:
            row.append(sent.count(word))
        matrix.append(row)
    matrix = np.array(matrix) # rows - words, colums - sentences

    # SVD
    u, e, vh = (np.linalg.svd(matrix))
    # slicing
    un = []
    for row in u:
        un.append(row[:k]) # first k colums
    un = np.transpose(np.array(un))
    vn = np.array(vh[:k]) # first k rows
```

ПОЛУЧЕНИЕ ТОЧЕК В К-МЕРНОМ СЕМАНТИЧЕСКОМ ПРОСТРАНСТВЕ

Взятие первых k столбцов матрицы U и k строк матрицы VT даёт нам **точки на пространстве, соответсвующие отдельным словам** (для U) и **предложениям** (для VT).

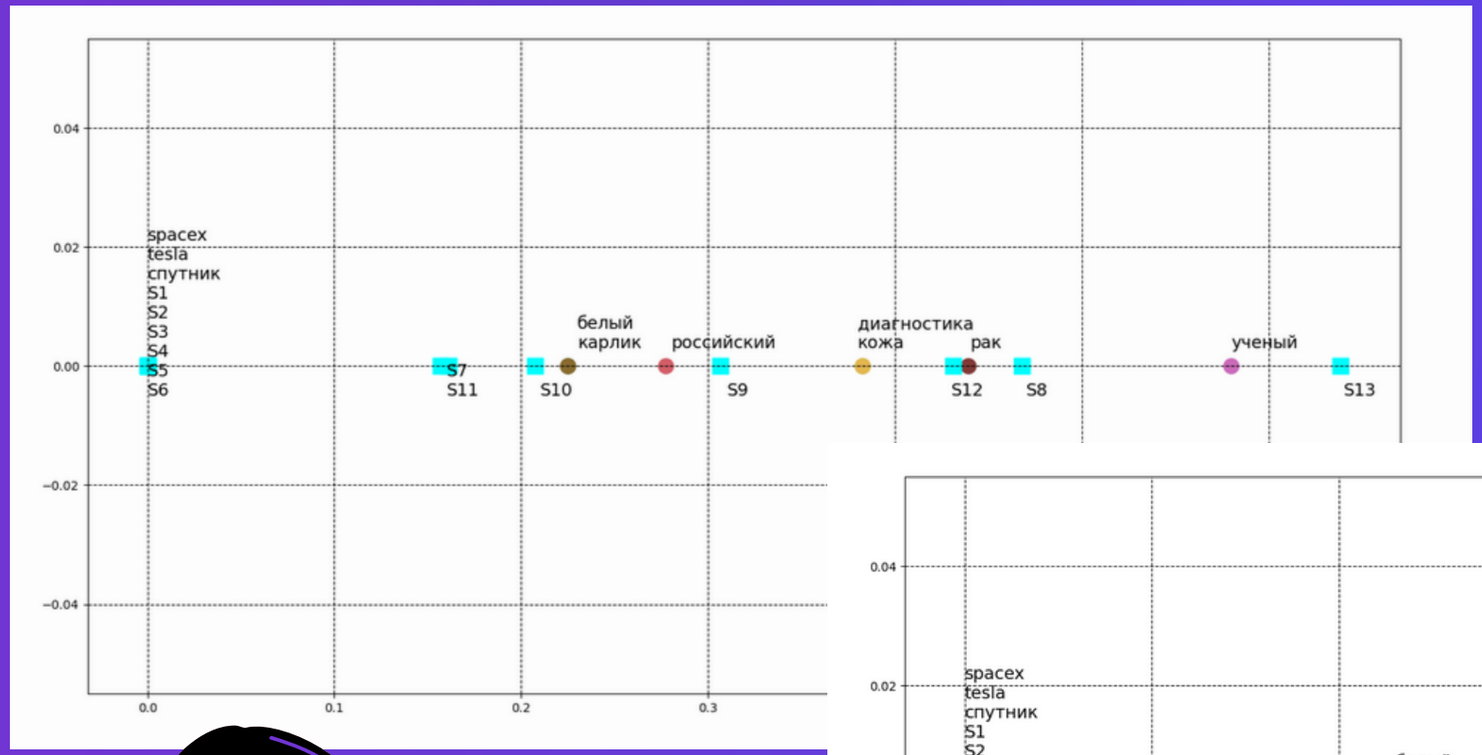


КЛАСТЕРИЗАЦИЯ

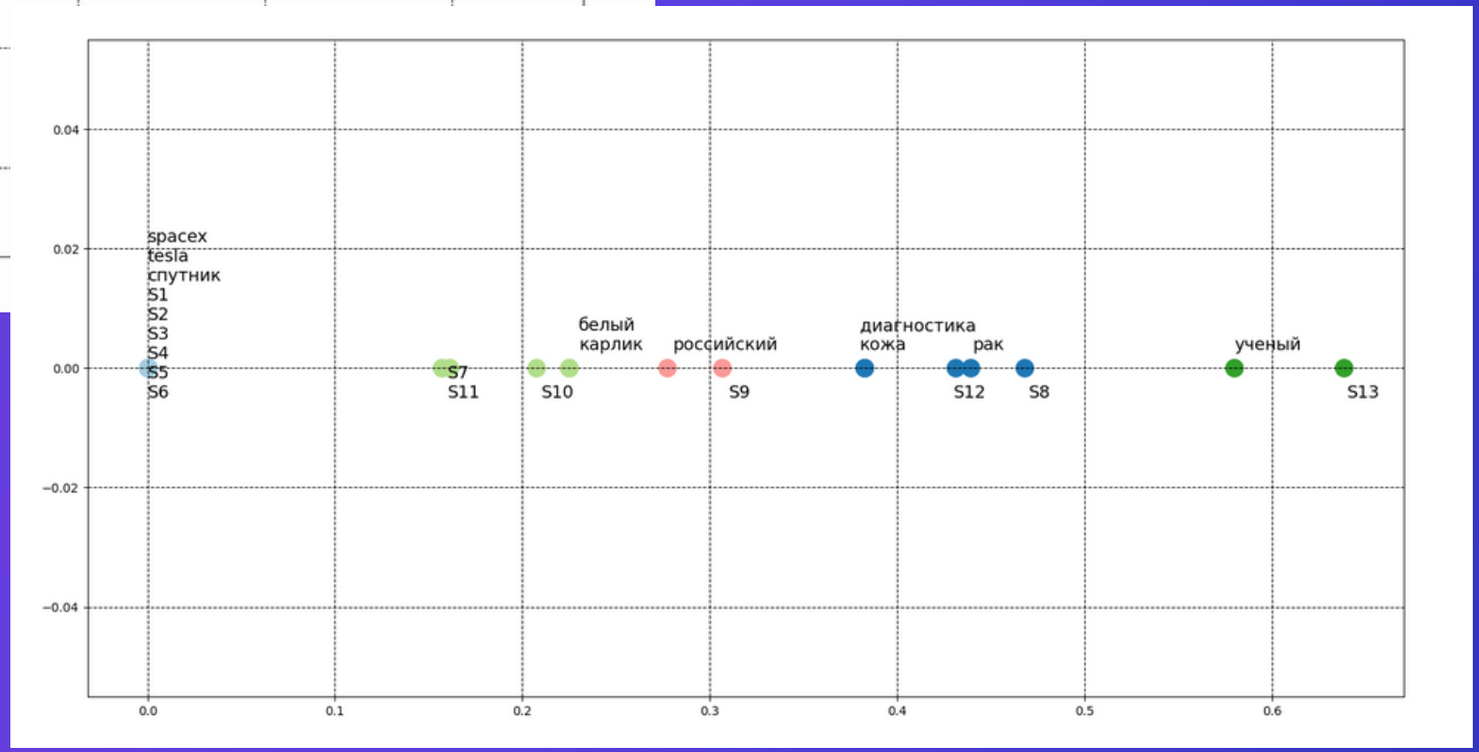
```
wpoints = list(zip(*un))
    spoints = list(zip(*vn))
    points = np.array(wpoints + spoints)
    kmeans = KMeans(n_clusters=n, random_state=0, n_init="auto").fit(points)

    if k <= 3: draw(k, un, vn, words, points, kmeans)
    else:
        print("\nU matrix:")
        for row in un:
            for element in row:
                print("{:5.2f}".format(element), end = " ")
            print()
        print("\nVT matrix:")
        for row in vn:
            for element in row:
                print("{:5.2f}".format(element), end = " ")
            print()
        print("\nClusters:")
        for p, label in zip(points, kmeans.labels_):
            point = list(map(lambda x: round(x, 2), p))
            print(f"{point} point in {label} cluster")
```

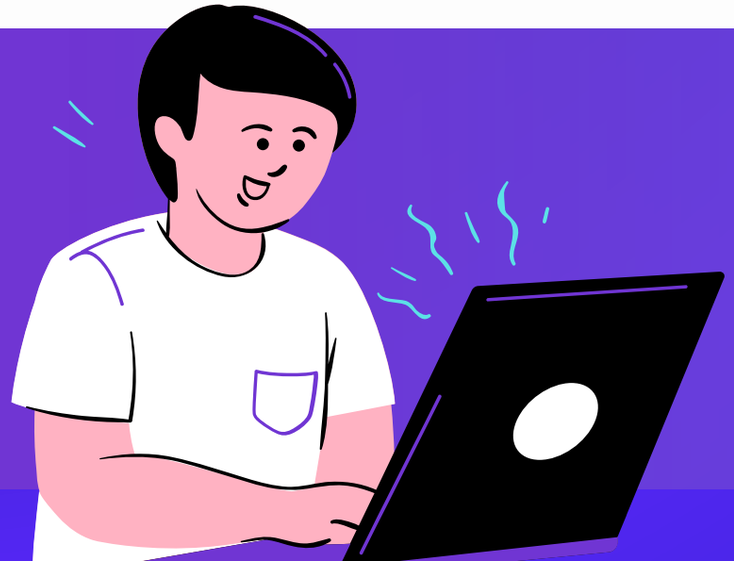
Для определения схожести мы можем использовать, например, уже известный нам **алгоритм k-means,** чтобы задать **разбиение точек** полученного пространства на n кластеров. Здесь мы используем библиотеку sklearn, которая его реализовывает.

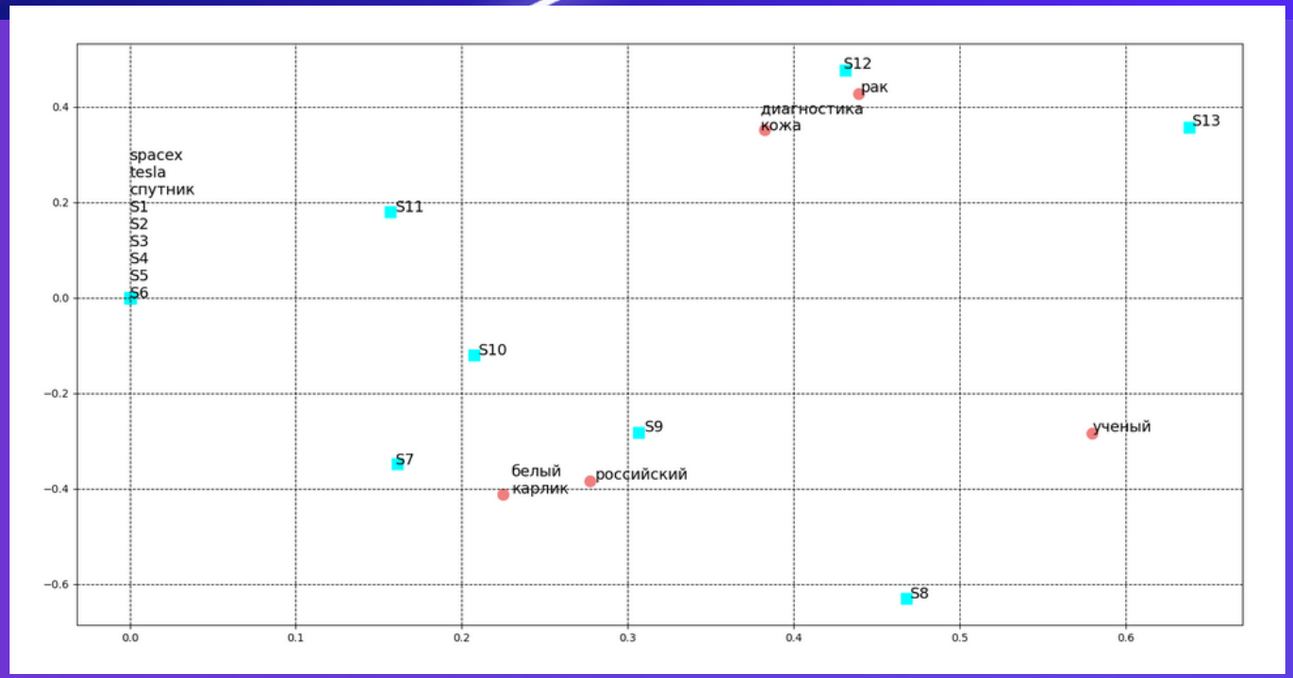
ПОЛУЧЕНИЕ ТОЧЕК В СЕМАНТИЧЕСКОМ ПРОСТРАНСТВЕ



1D РАЗБИЕНИЕ
НА 5 КЛАСТЕРОВ

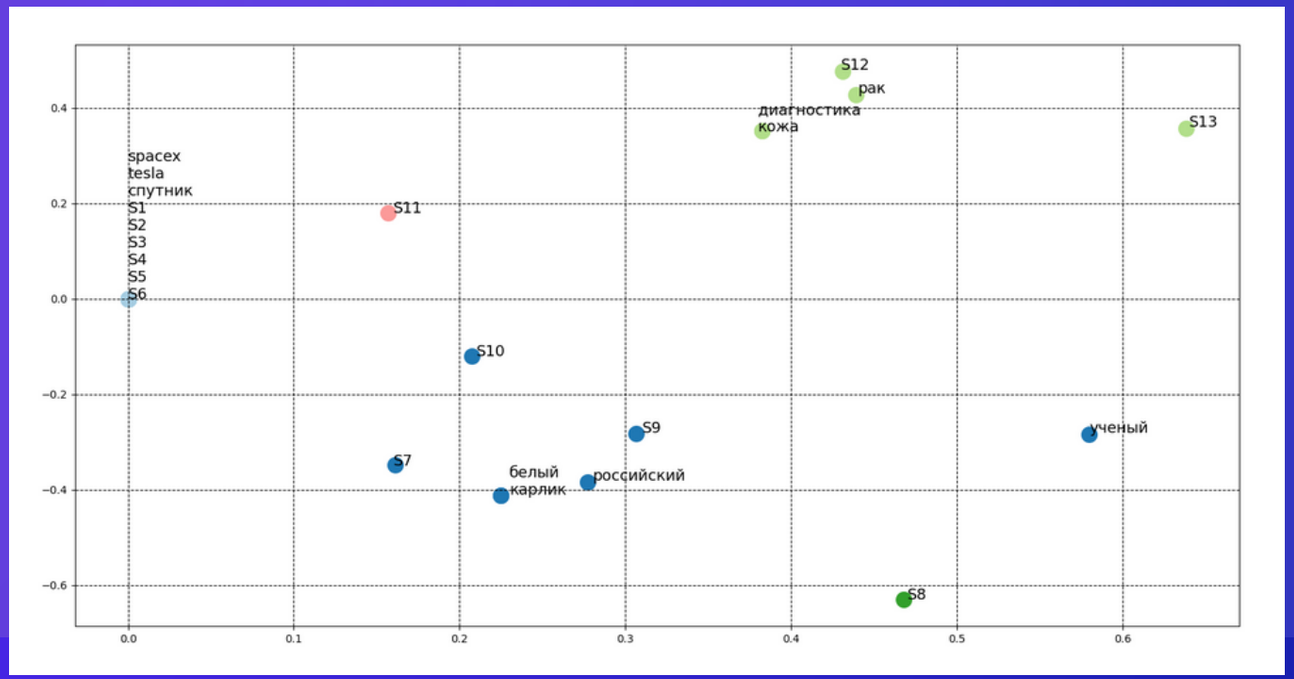




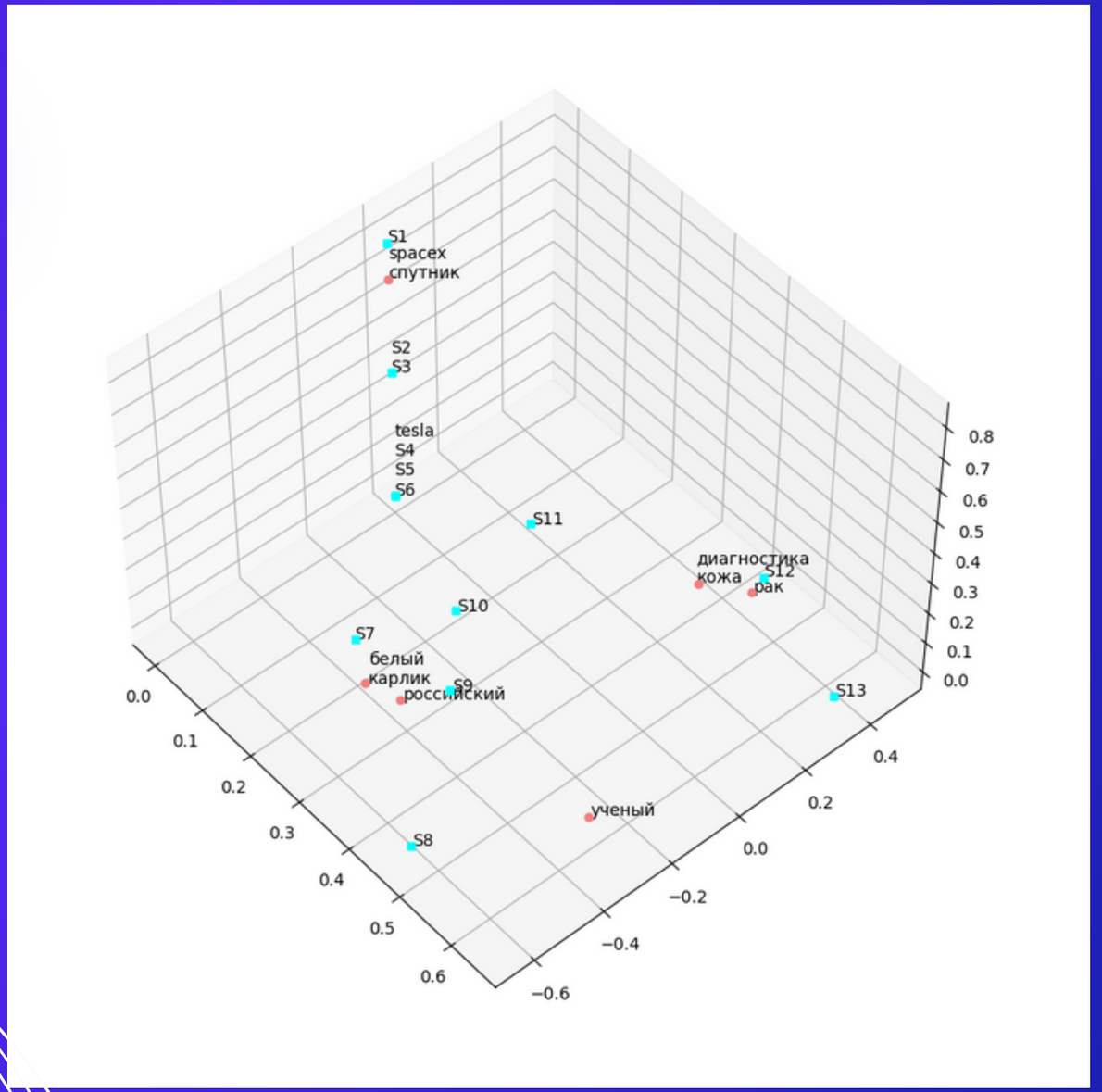


2D РАЗБИТИЕ НА 5 КЛАСТЕРОВ

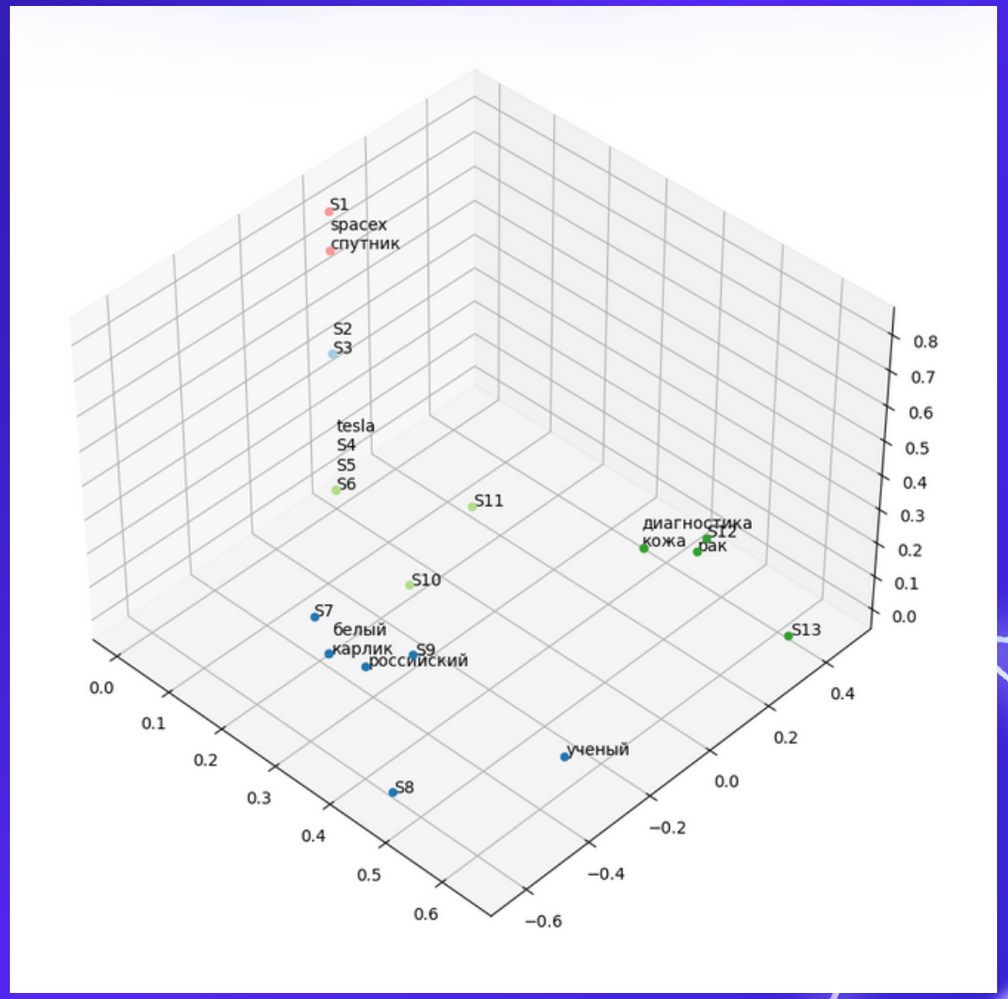
ПОЛУЧЕНИЕ ТОЧЕК В 2D



ПРЕОБРАЗОВАННЫЕ СЛОВА В 3D



5 КЛАСТЕРОВ В 3D



ПОЛУЧЕНИЕ ГРАФИКОВ

```
def draw(k, un, vn, words, points, kmeans):
    # plotting
    positions = {} # same points have same label
    cluster_positions = {}
    # 1d
    if k == 1:
        fig, ax = plt.subplots()
        # words
        for i in range(len(un[0])):
            x = un[0][i]
            y = 0
            ax.scatter(x, y, label=words[i], color=cm.tab20b(i/len(un[0])), s=150)
            if round(x, 2) in positions:
                positions[round(x, 2)] = positions[round(x, 2)] + "\n" + words[i]
            else:
                positions[round(x, 2)] = words[i]

        #sentences
        for j in range(len(vn[0])):
            x = vn[0][j]
            y = 0
            ax.scatter(x, y, label=f"S{j}", color='cyan', marker="s", s=150)
            if round(x, 2) in positions:
                positions[round(x, 2)] = positions[round(x, 2)] + "\n" + f"S{j+1}"
            else:
                positions[round(x, 2)] = f"S{j+1}"

        # annotating
        for pos in positions:
            if re.search("S\d+", positions[pos]):
                textpos = (pos, -0.005)
            else:
                textpos = (pos, 0.003)
            ax.annotate(positions[pos], (pos, 0), textpos, fontsize=14)
        ax.grid(color='k', linestyle='--')
```

```
    # clusters plotting
    fig, ax = plt.subplots()
    for p, label in zip(points, kmeans.labels_):
        ax.scatter(*p, 0, color=cm.Paired(label), s=200)
    for pos in positions:
        if re.search("S\d+", positions[pos]):
            textpos = (pos, -0.005)
        else:
            textpos = (pos, 0.003)
        ax.annotate(positions[pos], (pos, 0), textpos, fontsize=14)
    ax.grid(color='k', linestyle='--')
    plt.show()
    # 2d
    elif k == 2:
        fig, ax = plt.subplots()
        # words
        for i in range(len(un[0])):
            x = un[0][i]
            y = un[1][i] if k == 2 else 0
            ax.scatter(x, y, label=words[i], color='lightcoral', s=100)
            positions[(round(x, 2), round(y, 2))] = positions.get((round(x, 2), round(y, 2)), "") + "\n" + words[i]

        #sentences
        for j in range(len(vn[0])):
            x = vn[0][j]
            y = vn[1][j] if k == 2 else 0
            ax.scatter(x, y, label=f"S{j}", color='cyan', marker="s", s=100)
            positions[(round(x, 2), round(y, 2))] = positions.get((round(x, 2), round(y, 2)), "") + "\n" + f"S{j+1}"
        # annotating
        for pos in positions:
            ax.annotate(positions[pos], pos, fontsize=14)
        ax.grid(color='k', linestyle='--')
    # clusters plotting
        fig, ax = plt.subplots()
        for p, label in zip(points, kmeans.labels_):
            ax.scatter(*p, color=cm.Paired(label), s=200)
        for pos in positions:
            ax.annotate(positions[pos], pos, fontsize=14)
        ax.grid(color='k', linestyle='--')
        plt.show()
```

```
    # 3d
    elif k == 3:
        ax = plt.figure().add_subplot(projection='3d')
        #words
        for i in range(len(un[0])):
            x = un[0][i]
            y = un[1][i]
            z = un[2][i]
            ax.scatter(x, y, z, label=words[i], color='lightcoral')
            positions[(round(x, 2), round(y, 2), round(z, 2))] = positions.get((round(x, 2), round(y, 2), round(z, 2)), "") + "\n" + words[i]
```

```
        #sentences
        for j in range(len(vn[0])):
            x = vn[0][j]
            y = vn[1][j]
            z = vn[2][j]
            ax.scatter(x, y, z, label=f"S{j}", color='cyan', marker="s")
            positions[(round(x, 2), round(y, 2), round(z, 2))] = positions.get((round(x, 2), round(y, 2), round(z, 2)), "") + "\n" + f"S{j+1}"
        #annotating
        for pos in positions:
            ax.text(*pos, positions[pos])

        # clusters plotting
        ax = plt.figure().add_subplot(projection='3d')
        for p, label in zip(points, kmeans.labels_):
            ax.scatter(*p, color=cm.Paired(label))
        for pos in positions:
            ax.text(*pos, positions[pos])
        plt.show()

    if __name__ == "__main__":
        main()
```

Как оказалось, Латентно-семантический анализ **не сложен**.
Из графиков видно, что слова образуют независимые группы, и кластеризация успешно удалась!

Теперь, после прохождения нашей **мини-лекции**,
Вы можете **находить местоположения слов и статей** в пространстве и использовать эту информацию для **определения тематики большого массива данны**х.

GITHUB с проектом:
https://github.com/ventusjke/lab_pla